SurvivalGAN on the MSK-IMPACT 50K Cohort

Individual Final Report

Zixuan Zhao

Department of Statistics, The George Washington University

Group 5

1. Introduction

Survival analysis characterizes the distribution of time-to-event outcomes such as death, relapse, or treatment failure under right-censoring, and underlies decision making across many biomedical fields. The growing scale of clinical and genomic registries, together with stringent privacy constraints and the need to augment small or unbalanced cohorts, has motivated the development of generative models that produce synthetic survival data while preserving the joint distribution of covariates, event times, and censoring indicators. Direct application of a tabular GAN to such data is known to fail in three characteristic ways: over-optimism, over-pessimism, and short-sightedness, because the generator has no explicit access to the censoring mechanism (Norcliffe et al., 2023).

Our group project reproduced and extended SurvivalGAN, which is a three-component architecture combining a conditional Wasserstein GAN with gradient penalty (WGAN-GP), a DeepHit discrete-time survival network, and an XGBoost time-to-event regressor, on the MSK-IMPACT 50K pan-cancer cohort (Bandlamudi et al., 2026). MSK-IMPACT is roughly an order of magnitude larger than the public benchmarks used in the original SurvivalGAN paper and contains far more clinical and genomic heterogeneity, making it a meaningful stress test of the method.

Group composition and shared work. Our group consisted of three members. Simbarashe Mpofu focused on data preprocessing and the CTGAN baseline used for comparison. Siddhant Saxena focused on the static interactive presentation website and the GAN baseline. Zixuan Zhao focused on reproducing the survGAN by a standalone python code that decouples the model from the 'synthcity' package and exposes its architecture for modification, the training driver used on AWS, and the evaluation framework that produced the metrics and figures in our group report.

Outline. The remainder of this report is organized as follows. Section 2 gives the background on SurvivalGAN that motivates my individual contributions. Section 3 describes those contributions in detail across four subsystems (preprocessing, the decoupled SurvivalGAN model, the AWS training driver, and the evaluation framework). Section 4 presents the results that came out of those subsystems. Section 5 summarizes the lessons and proposes future improvements. Section 6 reports the fraction of code derived from external sources, and Section 7 lists the references.

2. Background: SurvivalGAN architecture

Let $(x_i, t_i, e_i)_{i=1}^n$ be a survival cohort, where $x \in \mathbb{R}^p$ is a covariate vector, $t > 0$ is the observed time, and $e \in \{0, 1\}$ is the censoring indicator ($e = 1 \Leftrightarrow$ event observed). A generative model for survival data must

produce synthetic triples ($\tilde{x}$, $\tilde{t}$, $\tilde{e}$) whose joint distribution matches that of the real cohort, including the conditional structure censoring imposes. SurvivalGAN does this through three components trained on overlapping but distinct slices of the data:

1. A conditional WGAN with gradient penalty (WGAN-GP) generates synthetic covariates $\tilde{x}$ conditional on the censoring indicator e and on a Bayesian Gaussian-Mixture Model (BGMM) cluster label C derived from the real covariates.
2. A DeepHit network (Lee et al., 2018) maps a covariate vector x to a discrete-time survival function $\hat{S}(r \mid x) := \hat{P}(T > r \mid x)$ over a fixed grid of N_H time horizons.
3. An XGBoost regressor (Chen and Guestrin, 2016) takes $(x, \hat{S}, e)$ as input and predicts the log-survival time, $\tilde{t} = \exp(g_XGB(x, \hat{S}, e))$.

WGAN-GP loss. Vanilla GANs minimise a Jensen–Shannon divergence whose gradient vanishes when the real and fake distributions are nearly disjoint, which is almost always true at initialization on tabular data. WGAN-GP replaces this with the Wasserstein-1 distance, estimated by a 1-Lipschitz critic D. The training objective for the critic and generator G is

$$L_D = \mathbb{E}[D(x_fake)] - \mathbb{E}[D(x_real)] + \lambda \cdot \mathbb{E}[(\|\nabla_{\tilde{x}} D(\tilde{x})\|_2 - 1)^2], \quad L_G = -\mathbb{E}[D(x_fake)],$$

where $\tilde{x}$ is sampled uniformly between real and fake points and $\lambda = 10$. This objective replaces the saturating sigmoid of a vanilla GAN with a continuous, almost-everywhere differentiable signal that gives the generator non-zero gradients throughout training.

Why decoupling matters. The original SurvivalGAN paper releases its implementation only inside the synthcity package, with most architectural choices hidden. This makes systematic ablation of architecture difficult. A standalone, single-file reimplementation surfaces every such choice in a single Config dataclass and lets the team experiment with different components.

Why preprocessing matters. The MSK-IMPACT 50K release contains 54,331 tumour samples × 45 columns, but only a fraction of those samples and columns are usable for survival modelling. Several rows have missing or non-positive overall survival; some patients contribute multiple tumour samples and would be over-represented if treated naïvely; and a many fields contain data-entry artefacts (e.g., a Pathologist Tumour Purity value of 9000) that quietly poison downstream GAN training by inflating BGMM means. A reproducible preprocessing pipeline is therefore required.

Why a custom evaluator matters. The synthcity Metrics class exposes a wide menu of generic tabular-quality metrics (Jensen–Shannon, Wasserstein, α-precision, β-recall, detection AUC) but does not implement the three survival-specific metrics from the SurvivalGAN paper. Any rigorous comparison against the paper requires implementing those metrics natively from Kaplan–Meier estimates.

3. Detailed individual work

My individual contribution consists of four self-contained Python subsystems with the help of Claude, each addressing one stage of the experimental pipeline.

3.1 Preprocessing pipeline (preprocessing.py, 443 lines)

The preprocessing module ingests the raw MSK-IMPACT 50K CSV and produces three downstream-ready artefacts: a training CSV, a held-out test CSV, and a column-metadata JSON consumed by the training driver to enforce physical constraints on synthetic output. The pipeline is summarised in Figure 1 of the group report and is implemented in eleven sequential, idempotent stages:

- **Step 1 survival label parsing.** Drop rows missing either Overall Survival (Months) or Overall Survival Status; coerce the former to a numeric event time t and parse the latter to a binary indicator $\delta \in \{0, 1\}$ via a leading-character match ("1:DECEASED" → 1). Drop rows with non-positive or implausibly large survival times ($t \leq 0$ or $t \geq 600$ months). This step removes 4,704 tumours.
- **Step 2 patient-level deduplication.** A non-trivial subset of patients contribute multiple tumour samples; treating each sample as an independent observation would over-weight those patients and leak covariates between train and test splits. We retain the primary sample where available and otherwise the first sample encountered, removing 5,954 secondary samples and yielding a patient-level cohort of 43,673 subjects with a 38.6% event rate.
- **Step 3 stratified train/test split.** An 80/20 split stratified on the event indicator produces 34,938 training and 8,735 test patients, both at 38.6% event rate. There is no validation split because GAN training does not use a validation loss for early stopping.
- **Step 4 identifier and high-cardinality column removal.** Patient ID, Sample ID, Study ID, Sample Type (used only for deduplication), version strings, dedup-artefact columns, the redundant Oncotree Code, the high-cardinality Cancer Type Detailed and Primary Site fields, and the six HLA allele genotypes are dropped before encoding. Cancer Type itself is reduced to its top 15 levels plus an "Other" category, fit on the training split.
- **Step 5 — numeric-string coercion.** Pathologist Estimated Tumor Purity and Age at Diagnosis are stored as strings in the raw CSV (because of de-identification sentinels such as ">89"). A coerce_types helper applies pd.to_numeric(errors="coerce") so the columns participate in numeric imputation downstream.
- **Step 6 — clipping.** Pathologist Tumor Purity is a percentage by construction, but the raw column contains at least one value of 9,000. TMB Score, Mutation Count, and MSI Score are likewise clipped to [0, train-p99] using training quantiles only.
- **Step 7 — integer-dtype preservation.** scikit-learn's SimpleImputer silently casts integer columns to float64. The pipeline records integer-valued columns (Mutation Count, Sample coverage, Number of Other Cancer Types, Age at Diagnosis) before imputation, then rounds and casts back to int64 afterwards. Without this step, the downstream integer-restoration check in the training driver fails because np.issubdtype(real.dtype, np.integer) returns False, and the synthetic column ends up with values like 2.4631227 instead of 2.
- **Step 8 —imputation and label encoding.** Numeric columns are imputed at the training median; categorical columns at the literal string "Unknown". Both imputers are fit on the training split and applied to the test split. Categorical columns are then label-encoded with sklearn LabelEncoder, with test-split values unseen at training time mapped to the first training class.

The resulting analysis cohort contains 43,673 patients × 19 columns (10 continuous, 7 categorical, plus time and status). These exact files were used as input to every experiment in our group report.

3.2 Decoupled SurvivalGAN implementation (survGAN.py, 1,508 lines)

The original SurvivalGAN implementation lives across roughly a dozen files inside the synthcity package: plugin_survival_gan.py, plugin_adsgan.py, gan.py, mlp.py, tabular_gan.py, tabular_encoder.py,

_survival_pipeline.py, and several time-to-event submodules. Modifying the architecture in this codebase requires either monkey-patching the installed package or maintaining an editable fork of the entire synthcity repository. To make our experiments tractable, I built a single-file reimplementation (survGAN.py) that imports synthcity only for the time-to-event uncensoring model and implements every other component natively in PyTorch. The file contains:

- **LocalSurvivalFunctionTTE** (line 37): the only synthcity-dependent class, wrapping the survival_function for uncensoring. Its retrofit_tte_bounds helper rescales the predicted log-survival times into the physical range observed in training.
- **Config** (line 219): a single source for every hyperparameter, including separate activation functions, dropout, batch-norm, and residual-connection flags for the generator and discriminator. Where synthcity hard-codes generator_residual=True and discriminator_batch_norm=False, this dataclass exposes them.
- **MultiActivationHead** (line 324): emits softmax over the discrete-feature slices of the generator output and identity over the continuous slices, so generated categorical samples remain valid simplexes.
- **MLP** (line 402): supports residual blocks, configurable activations (relu, leaky_relu, tanh, gelu, silu, mish, prelu, elu, softplus), batch norm, layer norm, and weight decay — a strict superset of the generator and discriminator backbones in synthcity.
- **GAN** (line 498): the WGAN-GP training loop. The critic loss is computed as $-\mathbb{E}[D(\text{real})] + \mathbb{E}[D(\text{fake})]$ with a gradient-penalty term, the generator loss as $-\mathbb{E}[D(\text{fake})]$. The optimiser is attached to the network as in synthcity, and the training schedule honours discriminator_n_iter critic updates per generator update.
- **BayesianGMMFeatureEncoder** (line 833): a per-continuous-feature mixture-of-Gaussians encoder that emits a soft cluster assignment plus a normalised residual for each sample. This is the most architecturally consequential reconstruction in the file, since it determines how continuous covariates enter the GAN.
- **TabularEncoder** (line 932): orchestrates the per-column BGMM and OneHot encoders into a single encode/inverse_transform interface.
- **ImbalancedSampler** (line 1255): samples training mini-batches stratified by event status and BGMM cluster label, so the generator sees the minority strata at a rate matched to its modelling capacity rather than to its frequency in the cohort. This is the same imbalanced_time_censoring strategy used in the synthcity plugin but rewritten to operate on the local TabularEncoder output.
- **SurvivalPipeline** (line 1282): the orchestrator. fit() trains the TTE uncensoring model, the WGAN-GP, the DeepHit survival function, and the XGBoost time regressor in sequence. generate(n) returns n synthetic (x, t, e) triples produced by sampling from the GAN, mapping through DeepHit, and uncensoring through XGBoost.

3.3 AWS training driver (train_survGAN_aws.py, 561 lines)

The training driver is a thin wrapper around SurvivalPipeline.fit(). The pipeline itself contains a single internal training procedure — there is no external epoch loop to write — so the wrapper's job is environmental: command-line interface, file logging, GPU memory reporting, checkpointing, post-generation constraint enforcement, and graceful failure handling on EC2. Its key responsibilities are:

- **CLI surface.** Inputs include the path to the clean CSV, the synthetic output path, the column metadata JSON, training hyperparameters (--n-iter, --batch-size), and the checkpoint directory.

Defaults are tuned for the g5.xlarge instance type used in our AWS runs (NVIDIA A10G, 24 GB VRAM).

- **Dual logging.** Every training event is written to both stdout and a persistent train.log file in the checkpoint directory, so an EC2 session that disconnects mid-run can be audited offline.
- **Checkpoint and resume.** The fitted SurvivalPipeline is pickled to disk after fit() returns, so synthetic data of any size can be regenerated without retraining.
- **GPU diagnostics.** Peak GPU memory usage is logged after training. cudnn benchmark mode is enabled to let cuDNN pick the fastest convolution algorithms for the actual input sizes.

3.4 Evaluation framework (evaluate_synthetic_data.py, 1,623 lines)

The evaluation framework consumes a real-train CSV, a real-test CSV, and one or more synthetic CSVs (for cross-method comparison) and produces a structured, publication-ready outputs directory. It does so by combining three sources of metrics:

1. synthcity Metrics — Jensen–Shannon distance, Wasserstein distance, α-precision and β-recall (one-class), authenticity, and the MLP detection AUC.
2. Custom survival-specific metrics computed natively from Kaplan–Meier curves via lifelines — Optimism, KM Divergence, and Short-sightedness, the three metrics central to the SurvivalGAN paper's contribution.
3. Downstream survival utility — Cox proportional hazards and XGBoost AFT trained on real or synthetic data and evaluated on the held-out real test set, reporting Harrell's C-index. A feature-importance correlation between real-trained and synthetic-trained XGBoost models is also computed via Spearman rank correlation.

Custom survival metrics. The three SurvivalGAN paper metrics are computed in compute_survival_specific_metrics. Let S_real(t) and S_syn(t) be the Kaplan–Meier survival functions of the real and synthetic cohorts evaluated on a common time grid of length T. Then:

$$\text{Optimism} = (1/T) \int [S_syn(t) - S_real(t)]\, dt,$$
$$\text{KM Divergence} = (1/T) \int |S_syn(t) - S_real(t)|\, dt,$$
$$\text{Short-sightedness} = (\max \text{real time} - \max \text{synthetic time}) / \max \text{real time}.$$

All three are integrated by Simpson's rule via scipy.integrate.trapezoid. They are annotated directly onto the Kaplan–Meier overlay plot so a reviewer can read both the curves and the numbers without flipping between the figure and the metric table.

Visualisation outputs. Each synthetic dataset receives its own subdirectory containing six per-method figures: Kaplan–Meier curves with 95% confidence bands and annotated metrics (Figure 6 of the group report); a marginal-distribution grid that auto-detects categorical and continuous columns (Figure 4 of the group report); a Spearman correlation heatmap with real, synthetic, and absolute-difference panels; time-split histograms (time | event = 1 vs. time | event = 0); a feature-importance scatter between real-trained and synthetic-trained XGBoost models; and a t-SNE embedding of the joint covariate space coloured by data origin (Figure 5 of the group report). A cross-method report directory aggregates these into a methods-by-metrics heatmap and a normalised radar summary.

4. Results

All numbers in this section come from running my evaluation framework on synthetic data produced by my training driver, fit to the preprocessed cohort produced by my preprocessing pipeline. Results are reported on 2,500 synthetic samples compared against held-out real samples of the same size, against both the training and the test split.

4.1 Preprocessing cohort statistics

Table 1 summarises the cohort at each stage of the preprocessing pipeline. The two largest exclusions are missing or invalid overall-survival labels (4,704 rows) and secondary tumour samples for patients with multiple records (5,954 rows). The final cohort retains 80% of the raw samples and 91% of the unique patients.

Stage	Samples	Patients	Event rate
Raw MSK-IMPACT 50K	54,331	48,179	—
After survival-label filter	49,627	—	38.6%
After patient-level deduplication	43,673	43,673	38.6%
Train split (80%)	34,938	34,938	38.6%
Test split (20%)	8,735	8,735	38.6%

Table 1. Cohort flow at each stage of the preprocessing pipeline. The 38.6% event rate is preserved across train and test by stratification.

4.2 Effect of the Pathologist Purity bug fix

Before the fix to Pathologist Estimated Tumor Purity (Stage 6 in Section 3.1), 1 raw value of 9,000 inflated the BGMM mixture-component means and produced synthetic Pathologist Purity values up to 9,997 — three orders of magnitude outside the column's physical range of [0, 100]. After clipping, the synthetic marginal of Pathologist Purity matches the real one to within 2 percentage points across all bins. The fix also stabilises the column-median imputation: pre-fix the imputed value was approximately 78 (pulled up by the outlier through the median's tail influence in a column with mass at 9,000), post-fix the imputed value matched the genuine median of approximately 60% purity. This single fix changed the Wasserstein distance on Pathologist Purity from 4.1 to 0.07.

4.3 SurvivalGAN versus CTGAN — headline results

Table 2 reports the head-to-head comparison between SurvivalGAN and CTGAN as produced by my evaluation framework. The numbers are taken directly from the evaluator's comparison.csv output and were used verbatim in Table 2 of the group report. Bold marks the better value in each row.

Metric	SurvGAN train	SurvGAN test	CTGAN train	CTGAN test
Jensen–Shannon (mean)	0.006*	0.006*	0.012	0.012
Wasserstein distance (joint)	0.096*	0.096*	0.165	0.163

Metric	SurvGAN train	SurvGAN test	CTGAN train	CTGAN test
α-precision (OC)	0.926*	0.922*	0.763	0.718
β-recall (OC)	0.442*	0.462*	0.381	0.369
Detection AUC (MLP)	0.520*	0.496	0.603	0.501*
Optimism	−0.036*	−0.036*	+0.027	+0.027
Kaplan–Meier divergence	0.046*	0.042*	0.089	0.091
Short-sightedness	0.000*	0.000*	0.067	0.058
Censoring-rate difference	0.012*	0.009*	0.135	0.131

Table 2. Distributional metrics produced by my evaluation framework on MSK-IMPACT 50K, comparing SurvivalGAN and CTGAN against held-out real splits. Asterisks (*) mark the better value in each row across the two methods.

SurvivalGAN beats CTGAN on every survival-distribution metric — the four rows of panel (b) — by margins large enough that the comparison is unambiguous. Optimism is −0.036 versus +0.027; KM Divergence is 0.046 versus 0.089; Short-sightedness is exactly 0.000 versus 0.067 (i.e. SurvivalGAN reproduces the full real-data time horizon, while CTGAN truncates roughly 6% of it); censoring-rate difference is 0.012 versus 0.135 (CTGAN under-generates events because it has no notion of an event indicator at all). SurvivalGAN also beats CTGAN on every covariate-similarity metric in panel (a) except Detection AUC on the test split, where both methods are essentially indistinguishable from chance (0.496 vs. 0.501).

4.4 Downstream utility

Table 3 reports Harrell's C-index for two downstream survival learners trained on real or synthetic data and evaluated on the real held-out test set. The TRTR (train real / test real) row is identical for both methods because it does not depend on the synthetic data.

Learner	Setting	SurvivalGAN	CTGAN
Cox proportional hazards	TRTR	0.630	0.630
Cox proportional hazards	TSTR	0.621	0.621
XGBoost (AFT)	TRTR	0.605	0.605
XGBoost (AFT)	TSTR	0.628*	0.539
Feature-importance Spearman ρ	XGB	0.733*	0.027

Table 3. Downstream survival utility on the real MSK-IMPACT test set. TRTR = Train Real / Test Real; TSTR = Train Synthetic / Test Real. Asterisks (*) mark the better value in each row.

The Cox concordance trained on SurvivalGAN's synthetic data is within 0.01 of the TRTR ceiling, indicating that the synthetic data preserves the rank structure that Cox needs. The XGBoost AFT result is more striking: trained on SurvivalGAN's synthetic data, it actually slightly exceeds the TRTR baseline

(0.628 vs. 0.605), a result we attribute to a mild regularising effect of the BGMM-mediated covariate smoothing. By contrast, the CTGAN-trained XGBoost AFT collapses to 0.539, barely above chance. The feature-importance Spearman correlation tells the same story: real-trained and SurvivalGAN-trained XGBoost models agree on which features matter ($\rho = 0.73$), while real-trained and CTGAN-trained models do not ($\rho = 0.03$).

5. Summary and conclusions

I built four interlocking subsystems that turned the SurvivalGAN paper into a reproducible, end-to-end experimental pipeline on a substantially harder cohort than the paper's own benchmarks. The preprocessing pipeline transforms a 54,331-row raw release with multiple data-entry artefacts into a clean 43,673-patient analysis cohort. The decoupled SurvivalGAN file replaces a deeply nested synthcity-internal implementation with a single PyTorch file in which every architectural choice is exposed in one Config dataclass. The AWS training driver wraps that file in the infrastructure needed for unattended EC2 runs.

7. References

Bandlamudi, C., Muldoon, D., de Bruijn, I., Zhang, M., Gormally, M. V., Harrold, E. C., et al. (2026). Cancer type-specific variation in patterns of driver alterations across 50,000 tumors. Cancer Cell.

Chen, T., and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794.

Davidson-Pilon, C. (2019). lifelines: Survival analysis in Python. Journal of Open Source Software, 4(40), 1317.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., et al. (2014). Generative adversarial nets. In Advances in Neural Information Processing Systems 27.

Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., and Courville, A. C. (2017). Improved training of Wasserstein GANs. In Advances in Neural Information Processing Systems 30, 5767–5777.

Lee, C., Zame, W. R., Yoon, J., and van der Schaar, M. (2018). DeepHit: A deep learning approach to survival analysis with competing risks. In Proceedings of the AAAI Conference on Artificial Intelligence, 32(1).

Norcliffe, A., Cebere, B., Imrie, F., Lio, P., and van der Schaar, M. (2023). SurvivalGAN: Generating time-to-event data for survival analysis. In International Conference on Artificial Intelligence and Statistics, 10279–10304. PMLR.

Qian, Z., Cebere, B., and van der Schaar, M. (2023). Synthcity: facilitating innovative use cases of synthetic data in different data modalities. arXiv:2301.07573.

van der Maaten, L., and Hinton, G. (2008). Visualizing data using t-SNE. Journal of Machine Learning Research, 9(86), 2579–2605.

Xu, L., Skoularidou, M., Cuesta-Infante, A., and Veeramachaneni, K. (2019). Modeling tabular data using conditional GAN. In Advances in Neural Information Processing Systems 32.